

EduGenie: Google Gemini Powered Learning Assistant

3. Project Design Phase

This document describes the architecture and module design documented for EduGenie, including the AI model allocation, folder structure, API layer, and frontend integration.

3.1 High-Level Architecture

EduGenie is organized as a frontend-to-FastAPI application. The HTML/CSS interface collects the learner's task and input, sends a POST request to the backend, and displays the returned result.

The backend exposes separate RESTful endpoints and routes each request to a corresponding functional module.

3.2 Model Allocation

Gemini 1.5 Pro via API: Q&A, summarization, quiz generation, and learning paths.

LaMini-Flan-T5-783M: concept explanation.

3.3 Repository / Folder Architecture

main.py — FastAPI application.

explanation_module.py — concept explanation logic.

qna.py — question answering.

quiz_module.py — quiz generation.

summary_module.py — summarization.

learning_path.py — learning recommendations.

templates/index.html — HTML frontend.

static/style.css — styling.

requirements.txt — Python dependencies.

3.4 Module Design

Explanation Module

Uses LaMini-Flan-T5 to provide concise, context-aware explanations that break complex topics into understandable language.

QnA Module

Uses Gemini 1.5 Pro for general knowledge and academic question answering.

Quiz Module

Builds a structured prompt, expects JSON, cleans Markdown code blocks with `clean_json_block`, and parses the cleaned result.

Summary Module

Uses Gemini capabilities to reduce long passages into concise, understandable summaries while retaining core information.

Learning Path Module

Uses `get_learning_recommendations` to request beginner-to-advanced concepts, organized by difficulty and supported by resources such as videos, articles, or books.

3.5 API Design

/qa

/explain

/quiz

/summarize

/learn/recommendations

3.6 Frontend Design

Task selection through a dropdown.

Text input through a textarea.

Submission through a button.

Returned content displayed below the input.

Responsive layout and styled controls.

Conclusion

The design is modular: each educational capability has its own backend logic while FastAPI acts as the common API layer and the HTML/CSS frontend provides a single learner-facing interface.

Source basis: EduGenie project document supplied with this request and the repository structure shown in the supplied reference image. Where the source did not specify an implementation detail, this documentation preserves the documented scope rather than inventing project facts.